Groth16 증명 생성 단계 (Proving Phase) 상세 분석

목차

1. 증명 생성 개요
2. 회로 합성 및 witness 할당
3. QAP witness 계산
4. 증명 구성 요소 계산
5. 영지식성 보장 메커니즘
6. 다중 스칼라 곱셈 최적화
7. 증명 재무작위화
8. 성능 분석 및 최적화

증명 생성 개요

증명 생성 단계는 실제 witness 값들을 사용하여 특정 계산이 올바르게 수행되었음을 증명하는 과정입니다. 이 단계에서는 설정 단계에서 생성된 증명 키와 실제 witness를 결합하여 3개의 그룹 원소로 구성된 간결한 증명을 생성합니다.

전체 워크플로우

```
pub fn create_proof_with_reduction<C>(
circuit: C,
```

```
pk: &ProvingKey<E>,

r: E::ScalarField, // 영지식성을 위한 무작위 값

s: E::ScalarField, // 영지식성을 위한 무작위 값

) -> R1CSResult<Proof<E>>

where

C: ConstraintSynthesizer<E::ScalarField>,

{

        let  prover_time = start_timer!(|| "Groth16::Prover");

        // 1. 제약 시스템 초기화

        let  cs = ConstraintSystem::new_ref();

        cs.set_optimization_goal(OptimizationGoal::Constraints);



        // 2. 회로 합성 (실제 witness와 함께)

        let  synthesis_time = start_timer!(|| "Constraint synthesis");

        circuit.generate_constraints(cs.clone())?;

        debug_assert!(cs.is_satisfied().unwrap()); // 제약 조건 만족 확인

        end_timer!(synthesis_time);



        // 3. 선형 결합 최적화

        let  lc_time = start_timer!(|| "Inlining LCs");

        cs.finalize();

        end_timer!(lc_time);
```

```
        // 4. QAP witness 계산
        let witness_map_time = start_timer!(|| "R1CS to QAP witness map");
        let h = QAP::witness_map::<E::ScalarField, D<E::ScalarField>>(cs.clone(
        end_timer!(witness_map_time);


        // 5. 증명 생성
        let prover = cs.borrow().unwrap();
        let proof = Self::create_proof_with_assignment(
            pk, r, s, &h,
            &prover.instance_assignment[1..], // 공개 입력 (첫 번째는 1)
            &prover.witness_assignment, // 비밀 witness
        )?;


        end_timer!(prover_time);
        Ok(proof)
    }
```

설정 단계와의 차이점

| 측면 | 설정 단계 | 증명 생성 단계 |

|------|----------|---------------|

| **목적** | 키 생성 | 증명 생성 |

| **입력** | 회로 구조만 | 회로 + 실제 witness |

| **모드** | `SynthesisMode::Setup` | 기본 모드 |

| **witness** | 없음 (구조만) | 실제 값들 |

| **검증** | 구조 검증 | 제약 조건 만족 검증 |

| **출력** | 증명/검증 키 | 증명 |

회로 합성 및 witness 할당

제약 시스템 초기화

```
let cs = ConstraintSystem::new_ref();
cs.set_optimization_goal(OptimizationGoal::Constraints);
// 설정 단계와 달리 SynthesisMode를 별도로 설정하지 않음 (기본 모드 사용)
```

기본 모드의 특징:

- **witness 저장**: 모든 변수에 실제 값 할당
- **제약 조건 검증**: 각 제약 조건이 실제로 만족되는지 확인
- **완전한 정보**: 증명 생성에 필요한 모든 정보 수집

회로 합성 과정

```
let synthesis_time = start_timer!(|| "Constraint synthesis");
circuit.generate_constraints(cs.clone())?;

// 중요: 모든 제약 조건이 만족되는지 확인
debug_assert!(cs.is_satisfied().unwrap());
```

```
end_timer!(synthesis_time);
```

제약 조건 만족 검증:

```
// ConstraintSystem 내부에서
pub fn is_satisfied(&self) -> Result<bool, SynthesisError> {
    for (i, constraint) in self.constraints.iter().enumerate() {
        let a_val = self.evaluate_lc(&constraint.a)?;
        let b_val = self.evaluate_lc(&constraint.b)?;
        let c_val = self.evaluate_lc(&constraint.c)?;
        // a * b = c 확인
        if a_val * b_val != c_val {
            eprintln!("Constraint {} is not satisfied", i);
            eprintln!(" a = {}, b = {}, c = {}", a_val, b_val, c_val
            eprintln!(" a * b = {}, expected = {}", a_val * b_val, c
            return Ok(false);
        }
    }
    Ok(true)
}
```

witness 할당 예시

간단한 곱셈 회로를 통해 witness 할당 과정을 살펴보겠습니다:

```
struct MultiplicationCircuit<F: Field> {
        a: Option<F>, // 비밀 입력
        b: Option<F>, // 비밀 입력
}


impl<F: Field> ConstraintSynthesizer<F> for MultiplicationCircuit<F> {
        fn  generate_constraints(self, cs: ConstraintSystemRef<F>) -> Result<(),
                // witness 변수 할당
                let  a_var = cs.new_witness_variable(|| {
                        self.a.ok_or(SynthesisError::AssignmentMissing)
                        })?;
                        let  b_var = cs.new_witness_variable(|| {
                        self.b.ok_or(SynthesisError::AssignmentMissing)
                })?;
                // 공개 입력 (결과값)
                let  c_var = cs.new_input_variable(|| {
                        let  a_val = self.a.ok_or(SynthesisError::AssignmentMiss
                        let  b_val = self.b.ok_or(SynthesisError::AssignmentMiss
                        Ok(a_val * b_val)
                })?;


                // 제약 조건: a * b = c
```

```
            cs.enforce_constraint(
                lc!() + a_var, // A = a
                lc!() + b_var, // B = b
                lc!() + c_var  // C = c
            )?;

            Ok(())
        }
}
```

변수 할당 과정:

1. **witness 변수**: `new_witness_variable` 로 비밀 값들 할당
2. **공개 입력**: `new_input_variable` 로 공개 값들 할당
3. **제약 조건**: `enforce_constraint` 로 관계식 정의
4. **검증**: 할당된 값들이 제약 조건을 만족하는지 확인

QAP witness 계산

QAP witness는 다음 방정식의 해 H(x)를 구하는 과정입니다:

```
A(x) * B(x) - C(x) = H(x) * Z(x)
```

witness 계산 과정

```rust
fn witness_map_from_matrices<F: PrimeField, D: EvaluationDomain<F>>(
matrices: &ConstraintMatrices<F>,
num_inputs: usize,
num_constraints: usize,
full_assignment: &[F],
) -> R1CSResult<Vec<F>> {
        let domain = D::new(num_constraints + num_inputs)?;
        let domain_size = domain.size();
        let zero = F::zero();

        // 1. A(x), B(x) 다항식을 평가 형태로 계산
        let mut a = vec![zero; domain_size];
        let mut b = vec![zero; domain_size];

        // 각 제약 조건에서 A, B 값 계산
        cfg_iter_mut!(a[..num_constraints])
        .zip(cfg_iter_mut!(b[..num_constraints]))
        .zip(cfg_iter!(&matrices.a))
        .zip(cfg_iter!(&matrices.b))
        .for_each(|(((a_val, b_val), a_constraint), b_constraint)| {
                *a_val = evaluate_constraint(a_constraint, full_assignment);
                *b_val = evaluate_constraint(b_constraint, full_assignment);
```

```
});

// 공개 입력 부분 설정
a[num_constraints..num_constraints + num_inputs]
.clone_from_slice(&full_assignment[..num_inputs]);

// 2. 계수 도메인에서 평가 도메인으로 변환 (IFFT)
domain.ifft_in_place(&mut a);
domain.ifft_in_place(&mut b);

// 3. 코셋에서 FFT (곱셈을 위해)
let coset_domain = domain.get_coset(F::GENERATOR).unwrap();
coset_domain.fft_in_place(&mut a);
coset_domain.fft_in_place(&mut b);

// 4. A(x) * B(x) 계산 (점별 곱셈)
let mut ab = domain.mul_polynomials_in_evaluation_domain(&a, &b);
drop(a); drop(b); // 메모리 절약

// 5. C(x) 계산
let mut c = vec![zero; domain_size];
cfg_iter_mut!(c[..num_constraints])
.enumerate()
```

```
        .for_each(|(i, c_val)| {
                *c_val = evaluate_constraint(&matrices.c[i], full_assignment);
        });


        domain.ifft_in_place(&mut  c);
        coset_domain.fft_in_place(&mut  c);


        // 6. H(x) = (A(x) * B(x) - C(x)) / Z(x) 계산
        let  vanishing_polynomial_over_coset = domain
        .evaluate_vanishing_polynomial(F::GENERATOR)
        .inverse()
        .unwrap();
        cfg_iter_mut!(ab).zip(c).for_each(|(ab_i, c_i)| {
                *ab_i -= &c_i; // A*B - C
                *ab_i *= &vanishing_polynomial_over_coset; // / Z
        });


        // 7. 계수 형태로 변환
        coset_domain.ifft_in_place(&mut  ab);
        Ok(ab)
}
```

제약 조건 평가 최적화

```rust
pub fn evaluate_constraint<F: PrimeField>(
terms: &[(F, usize)], // (계수, 변수 인덱스) 쌍들
assignment: &[F], // witness 할당
) -> F {
        #[cfg(feature = "parallel")]
        if terms.len() < 100 {
                serial_evaluate_constraint(terms, assignment)
        } else {
                // 병렬 처리로 성능 향상
                terms.par_iter()
                .map(|(coeff, index)| {
                        let val = assignment[*index];
                        if coeff.is_one() {
                                val // 곱셈 생략
                        } else {
                                val * coeff
                        }
                })
        .sum()
}
#[cfg(not(feature = "parallel"))]
serial_evaluate_constraint(terms, assignment)
```

```
}


fn serial_evaluate_constraint<F: PrimeField>(
terms: &[(F, usize)],
assignment: &[F]
) -> F {
        let mut sum = F::zero();
        // CPU 벡터화를 위해 청크 단위로 처리
        for chunk in terms.chunks(4) {
                let chunk_sum = chunk.iter()
                .map(|(coeff, index)| {
                        let val = assignment[*index];
                        if coeff.is_one() {
                                val
                        } else {
                                val * coeff
                        }
                })
                .sum::<F>();
        sum += chunk_sum;
        }
        sum
```

```
}
```

최적화 기법:

- **병렬 처리**: 큰 제약 조건에 대해 Rayon 사용
- **특수 케이스**: 계수가 1인 경우 곱셈 생략
- **청크 처리**: CPU 벡터화를 위한 작은 단위 처리
- **조기 종료**: 임계값 기반 병렬/순차 처리 선택

FFT 기반 다항식 연산의 상세 분석

1. 도메인 변환:

```
// 계수 → 평가 (IFFT)
domain.ifft_in_place(&mut a);
domain.ifft_in_place(&mut b);
```

- **목적**: 다항식 계수를 평가 점들에서의 값으로 변환
- **효율성**: O(n log n) 시간에 변환

2. 코셋 변환:

```
// 코셋에서 평가 (FFT)
let coset_domain = domain.get_coset(F::GENERATOR).unwrap();
coset_domain.fft_in_place(&mut a);
coset_domain.fft_in_place(&mut b);
```

- **목적**: 소멸 다항식으로 나누기 위해 생성원의 배수에서 평가

- **수학적 의미**: Z(g*ω^i) ≠ 0이므로 나눗셈 가능

3. 점별 곱셈:

```
let mut ab = domain.mul_polynomials_in_evaluation_domain(&a, &b);
```

- **효율성**: 평가 형태에서는 다항식 곱셈이 점별 곱셈으로 단순화
- **시간 복잡도**: O(n) (계수 형태에서는 $O(n^2)$)

증명 구성 요소 계산

전체 계산 과정

```
fn create_proof_with_assignment(
pk: &ProvingKey<E>,
r: E::ScalarField, // 영지식성을 위한 무작위 값
s: E::ScalarField, // 영지식성을 위한 무작위 값
h: &[E::ScalarField], // QAP witness
input_assignment: &[E::ScalarField], // 공개 입력
aux_assignment: &[E::ScalarField], // 비밀 witness
) -> R1CSResult<Proof<E>> {
        // === C 계산 준비 ===
        let c_acc_time = start_timer!(|| "Compute C");
        // H(τ) * t(τ) / δ 부분 계산
        let h_assignment = cfg_into_iter!(h)
        .map(|s| s.into_bigint())
```

```
.collect::<Vec<_>>();
let  h_acc = E::G1::msm_bigint(&pk.h_query, &h_assignment);
drop(h_assignment);


// L 부분 계산 (witness 변수들에 대한 선형 결합)
let  aux_assignment_bigint = cfg_iter!(aux_assignment)
.map(|s| s.into_bigint())
.collect::<Vec<_>>();
let  l_aux_acc = E::G1::msm_bigint(&pk.l_query, &aux_assignment_bigint);


// r * s * δ 부분
let  r_s_delta_g1 = pk.delta_g1 * (r * s);
end_timer!(c_acc_time);


// 전체 assignment 구성 (공개 입력 + witness)
let  input_assignment_bigint = input_assignment
.iter()
.map(|s| s.into_bigint())
.collect::<Vec<_>>();
let  assignment = [&input_assignment_bigint[..], &aux_assignment_bigint[
drop(aux_assignment_bigint);


// === A 계산 ===
```

```
let  a_acc_time = start_timer!(|| "Compute A");
let  r_g1 = pk.delta_g1.mul(r);
let  g_a = Self::calculate_coeff(r_g1, &pk.a_query, pk.vk.alpha_g1, &ass
let  s_g_a = g_a * &s; // A * s (C 계산용)
end_timer!(a_acc_time);


// === B 계산 (G1에서) ===
let  g1_b = if !r.is_zero() {
        let  b_g1_acc_time = start_timer!(|| "Compute B in G1");
        let  s_g1 = pk.delta_g1.mul(s);
        let  g1_b = Self::calculate_coeff(s_g1, &pk.b_g1_query, pk.beta_
        end_timer!(b_g1_acc_time);
        g1_b
} else {
        E::G1::zero()
};


// === B 계산 (G2에서) ===
let  b_g2_acc_time = start_timer!(|| "Compute B in G2");
let  s_g2 = pk.vk.delta_g2.mul(s);
let  g2_b = Self::calculate_coeff(s_g2, &pk.b_g2_query, pk.vk.beta_g2, &
let  r_g1_b = g1_b * &r; // B * r (C 계산용)
drop(assignment);
```

```
        end_timer!(b_g2_acc_time);


        // === C 최종 계산 ===
        let  c_time = start_timer!(|| "Finish C");
        let  mut  g_c = s_g_a; // A * s
        g_c += &r_g1_b; // + B * r
        g_c -= &r_s_delta_g1; // - r * s * δ
        g_c += &l_aux_acc; // + L(witness)
        g_c += &h_acc; // + H(τ) * t(τ) / δ
        end_timer!(c_time);


        Ok(Proof {
                a: g_a.into_affine(),
                b: g2_b.into_affine(),
                c: g_c.into_affine(),
        })
}
```

A 원소 계산 상세 분석

```
// A = α + Σ(a_i * u_i) + r * δ
let  r_g1 = pk.delta_g1.mul(r); // r * δ * G
```

```
let g_a = Self::calculate_coeff(r_g1, &pk.a_query, pk.vk.alpha_g1, &assignment)
```

수학적 구성:

1. **구조적 부분**: `α * G` (검증 키에서)
2. **witness 부분**: `Σ(a_i * u_i) * G` (MSM으로 계산)
3. **무작위화 부분**: `r * δ * G` (영지식성 보장)

B 원소 계산 상세 분석

G1에서의 B 계산 (C 원소용):

```
let g1_b = if !r.is_zero() {
let s_g1 = pk.delta_g1.mul(s);
Self::calculate_coeff(s_g1, &pk.b_g1_query, pk.beta_g1, &assignment)
} else {
E::G1::zero()
};
```

G2에서의 B 계산 (실제 증명용):

```
let s_g2 = pk.vk.delta_g2.mul(s);
let g2_b = Self::calculate_coeff(s_g2, &pk.b_g2_query, pk.vk.beta_g2, &assignme
```

이중 계산의 이유:

- **G1 버전**: C 원소에서 `B * r` 항 계산용
- **G2 버전**: 페어링 검증에서 사용할 실제 B 원소

C 원소 계산 상세 분석

C 원소는 가장 복잡한 구성 요소로, 다음 5개 항의 합으로 구성됩니다:

```
C = Σ(c_i * u_i) + H(τ) * t(τ) / δ + A * s + B * r - r * s * δ
```

각 항의 의미:

1. **`l_aux_acc`** : `Σ(c_i * u_i)` - witness와 C 다항식의 선형 결합
2. **`h_acc`** : `H(τ) * t(τ) / δ` - QAP 다항식 나눗셈의 몫
3. **`s_g_a`** : `A * s` - A와 s의 교차 항
4. **`r_g1_b`** : `B * r` - B와 r의 교차 항
5. **`-r_s_delta_g1`** : `-r * s * δ` - 페어링 방정식 균형 조정

계수 계산 헬퍼 함수

```
fn calculate_coeff<G: AffineRepr>(
initial: G::Group, // 초기값 (r*δ 또는 s*δ)
query: &[G], // 쿼리 벡터
vk_param: G, // 검증 키 매개변수 (α 또는 β)
assignment: &[<G::ScalarField as PrimeField>::BigInt], // witness 할당
) -> G::Group
where
G::Group: VariableBaseMSM<MulBase = G>,
{
let el = query[0]; // 첫 번째 원소
```

```
// MSM: Multi-Scalar Multiplication

let acc = G::Group::msm_bigint(&query[1..], assignment);



let mut res = initial; // r*δ 또는 s*δ

res.add_assign(&el); // + query[0]

res += &acc; // + Σ(assignment[i] * query[i+1])

res.add_assign(&vk_param); // + α 또는 β



res

}
```

MSM 최적화:

- **Pippenger 알고리즘**: 대규모 MSM에 최적화된 알고리즘
- **윈도우 방법**: 지수를 작은 윈도우로 나누어 처리
- **배치 처리**: 여러 MSM을 동시에 처리

영지식성 보장 메커니즘

무작위 값 생성

```
pub fn create_random_proof_with_reduction<C>(

circuit: C,

pk: &ProvingKey<E>,

rng: &mut impl Rng,
```

```
) -> R1CSResult<Proof<E>>

where

C: ConstraintSynthesizer<E::ScalarField>,

{

// 영지식성을 위한 무작위 값 생성

let  r = E::ScalarField::rand(rng);

let  s = E::ScalarField::rand(rng);



Self::create_proof_with_reduction(circuit, pk, r, s)

}
```

무작위 값의 역할:

- **r**: A와 C 원소에 무작위성 추가
- **s**: B와 C 원소에 무작위성 추가
- **독립성**: r과 s는 서로 독립적으로 생성

무작위화 메커니즘

A 원소 무작위화:

```
A = α + Σ(a_i * u_i) + r * δ
```

- `r * δ` 항이 실제 witness 값들을 마스킹
- 검증자는 `r` 을 알 수 없으므로 witness 정보 추출 불가

B 원소 무작위화:

```
B = β + Σ(b_i * u_i) + s * δ
```

- `s * δ` 항이 B 원소를 무작위화
- A와 독립적인 무작위성 제공

C 원소의 교차 항:

```
C = ... + A * s + B * r - r * s * δ
```

- **`A * s`** : A의 무작위성이 C에 전파
- **`B * r`** : B의 무작위성이 C에 전파
- **`-r * s * δ`** : 페어링 방정식의 균형 유지

영지식성의 수학적 증명

시뮬레이터 존재성:

```
// 실제 witness 없이도 동일한 분포의 증명 생성 가능
fn simulate_proof(
vk: &VerifyingKey<E>,
public_inputs: &[E::ScalarField],
rng: &mut impl Rng,
) -> Proof<E> {
        // 무작위 값들로 증명 구성
        let r = E::ScalarField::rand(rng);
        let s = E::ScalarField::rand(rng);
```

```
        // 실제 증명과 구별 불가능한 증명 생성

        // (단, 페어링 검증은 실패)

}
```

다중 스칼라 곱셈 최적화

MSM 알고리즘 선택

```
impl<G: CurveGroup> VariableBaseMSM for G {

        fn  msm_bigint(bases: &[G::Affine], scalars: &[BigInt]) -> G {

                match  scalars.len() {

                        0 => G::zero(),

                        1 => bases[0] * scalars[0],

                        _  if  scalars.len() < 32 => naive_msm(bases, scalars),

                        _ => pippenger_msm(bases, scalars),

                }

        }

}
```

알고리즘 선택 기준:

- **작은 크기 (< 32)**: 단순한 이중 곱셈
- **중간 크기**: 윈도우 방법
- **큰 크기**: Pippenger 알고리즘

Pippenger 알고리즘 구현

```
fn pippenger_msm<G: CurveGroup>(
bases: &[G::Affine],
scalars: &[BigInt]
) -> G {
        let window_size = optimal_window_size(scalars.len());
        let num_windows = (256 + window_size - 1) / window_size;
        let mut result = G::zero();
        for window in (0..num_windows).rev() {
                // 이전 결과를 window_size만큼 시프트
                for _ in 0..window_size {
                        result = result.double();
                }
                // 현재 윈도우에서 기여도 계산
                let mut buckets = vec![G::zero(); (1 << window_size) - 1];
                for (base, scalar) in bases.iter().zip(scalars.iter()) {
                        let window_val = extract_window(scalar, window, window_
                        if window_val != 0 {
                                buckets[window_val - 1] += base;
                        }
                }
                // 버킷들을 효율적으로 합산
```

```
            let window_sum = sum_buckets(&buckets);
            result += &window_sum;
        }
        result
    }
```

메모리 접근 패턴 최적화

```
// 캐시 친화적인 데이터 레이아웃
#[repr(C, align(64))] // 캐시 라인 정렬
struct AlignedScalar([u64; 4]);

// 프리페칭을 통한 성능 향상
fn prefetch_bases(bases: &[G::Affine], indices: &[usize]) {
        for &idx in indices.iter().take(8) { // 8개씩 미리 로드
            unsafe {
                std::arch::x86_64::_mm_prefetch(
                    &bases[idx] as *const _ as *const i8,
                    std::arch::x86_64::_MM_HINT_T0
                );
            }
        }
    }
```

증명 재무작위화

재무작위화 알고리즘

```
pub fn rerandomize_proof(
vk: &VerifyingKey<E>,
proof: &Proof<E>,
rng: &mut impl Rng,
) -> Proof<E> {
        // 0이 아닌 무작위 값들 생성
        let (mut r1, mut r2) = (E::ScalarField::zero(), E::ScalarField::zero()
        while r1.is_zero() || r2.is_zero() {
        r1 = E::ScalarField::rand(rng);
        r2 = E::ScalarField::rand(rng);
        }

        // 재무작위화 공식 적용
        // A' = (1/r₁ ) * A
        let new_a = proof.a.mul(r1.inverse().unwrap());
        // B' = r₁ * B + r₁ * r₂ * δ * H
        let new_b = proof.b.mul(r1) + &vk.delta_g2.mul(r1 * &r2);
        // C' = C + r₂ * A
        let new_c = proof.c + proof.a.mul(r2).into_affine();
```

```
        Proof {
                a: new_a.into_affine(),
                b: new_b.into_affine(),
                c: new_c.into_affine(),
        }
}
```

재무작위화의 수학적 정확성

페어링 방정식 보존:

```
원래: e(A, B) = e(α, β) * e(L_ic, γ) * e(C, δ)

재무작위화 후:
e(A', B') = e((1/r₁ )*A, r₁ *B + r₁ *r₂ *δ*H)
= e(A, B) * e(A, r₂ *δ*H)
= e(A, B) * e(r₂ *A, δ*H)
e(α, β) * e(L_ic, γ) * e(C', δ)
= e(α, β) * e(L_ic, γ) * e(C + r₂ *A, δ)
= e(α, β) * e(L_ic, γ) * e(C, δ) * e(r₂ *A, δ)

따라서 두 식이 같음을 확인할 수 있음
```

재무작위화의 활용

프라이버시 향상:

- 동일한 statement에 대해 다른 증명 생성
- 증명 간 연결성 제거
- 트랜잭션 프라이버시 보호

배치 처리:

```
pub fn batch_rerandomize(
vk: &VerifyingKey<E>,
proofs: &[Proof<E>],
rng: &mut impl Rng,
) -> Vec<Proof<E>> {
        proofs.iter()
        .map(|proof| Self::rerandomize_proof(vk, proof, rng))
        .collect()
}
```

성능 분석 및 최적화

시간 복잡도 분석

| 단계 | 시간 복잡도 | 주요 연산 |

|------|----------|----------|

| 회로 합성 | O(n) | 제약 조건 생성 |

| QAP witness | O(n log n) | FFT 연산 |

| A 계산 | O(n) | MSM |

| B 계산 | O(n) | MSM (G1 + G2) |

| C 계산 | O(n) | MSM + 덧셈 |

| **전체** | **O(n log n)** | **FFT 지배적** |

메모리 사용량 분석

```
// 주요 메모리 사용 구성 요소
struct MemoryUsage {
        constraint_system: usize, // O(n) - 제약 조건들
        qap_witness: usize, // O(n) - H(x) 다항식
        assignment_vectors: usize, // O(n) - witness 할당
        intermediate_results: usize, // O(n) - 중간 계산 결과
}

// 총 메모리 사용량: 약 4n * field_size
```

병렬 처리 최적화

```
// 병렬 제약 조건 평가
cfg_iter_mut!(a[..num_constraints])
.zip(cfg_iter_mut!(b[..num_constraints]))
.zip(cfg_iter!(&matrices.a))
```

```
.zip(cfg_iter!(&matrices.b))
.for_each(|(((a_val, b_val), a_constraint), b_constraint)| {
*a_val = evaluate_constraint(a_constraint, full_assignment);
*b_val = evaluate_constraint(b_constraint, full_assignment);
});
```

병렬화 효과:

- **제약 조건 평가**: 약 2-4배 성능 향상
- **MSM 연산**: 약 3-6배 성능 향상
- **FFT 연산**: 약 2-3배 성능 향상

실제 성능 벤치마크

```
// MiMC 해시 (322 라운드) 기준
const MIMC_ROUNDS: usize = 322;

// 성능 측정 결과 (BLS12-377, 단일 스레드)
struct BenchmarkResults {
        circuit_synthesis: Duration, // ~1ms
        qap_witness: Duration, // ~5ms
        proof_generation: Duration, // ~15ms
        total_proving: Duration, // ~21ms
        verification: Duration, // ~2ms
```

```
}
```

최적화 기법들:

1. **조기 메모리 해제**: 불필요한 데이터 즉시 해제
2. **배치 MSM**: 여러 MSM을 동시에 처리
3. **캐시 최적화**: 메모리 접근 패턴 최적화
4. **SIMD 활용**: 벡터 연산을 통한 가속화

증명 생성 단계는 Groth16의 핵심 기능을 구현하는 부분으로, 수학적 정확성과 계산 효율성을 모두 고려하여 설계되었습니다. 특히 영지식성 보장과 성능 최적화 사이의 균형을 잘 맞추고 있으며, 실제 애플리케이션에서 요구되는 모든 기능을 제공합니다.